Sesión 5

Manipulación de Datos y Lógica de Conjuntos

Carlos César Sánchez Coronel

BASES DE DATOS

Contenido general

1. Introducción y Sublenguajes SQL
2. Proceso Interno de una Consulta
3. Joins: Combinando Tablas
4. Subconsultas
5. CTEs
6. Buenas Prácticas y Optimización
7. Hardware y Breve Mirada a NoSQL
8. Ejercicios Resueltos
9. Resumen

Contenido de la sección

1. Introducción y Sublenguajes SQL
2. Proceso Interno de una Consulta
3. Joins: Combinando Tablas
4. Subconsultas
5. CTEs
6. Buenas Prácticas y Optimización
7. Hardware y Breve Mirada a NoSQL
8. Ejercicios Resueltos
9. Resumen

Sublenguajes SQL

Clasificación

- **DDL** (Data Definition Language): CREATE, ALTER, DROP, TRUNCATE.
- **DML** (Data Manipulation Language): SELECT, INSERT, UPDATE, DELETE, MERGE.
- **DCL** (Data Control Language): GRANT, REVOKE.
- **TCL** (Transaction Control Language): BEGIN, COMMIT, ROLLBACK, SAVEPOINT.

DDL: Creación y modificación de estructuras

CREATE TABLE

```sql
CREATE TABLE clientes (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE,
    ciudad VARCHAR(50),
    fecha_registro DATE DEFAULT CURRENT_DATE
);
```

ALTER y DROP

```sql
ALTER TABLE clientes ADD COLUMN telefono VARCHAR(20);
DROP TABLE clientes; -- Cuidado!
```

DML: Manipulación de datos

INSERT

```
INSERT INTO clientes (nombre, email, ciudad)
VALUES ('Carlos Ruiz', 'carlos@mail.com', '
    Valencia');
```

UPDATE

```
UPDATE clientes SET ciudad = 'Sevilla'
WHERE id = 5;
```

DELETE

```
DELETE FROM clientes WHERE email IS NULL;
```

MERGE (UPSERT en PostgreSQL)

```
INSERT INTO productos (id, nombre, precio)
VALUES (1, 'Laptop', 1200)
ON CONFLICT (id) DO UPDATE SET
    nombre = EXCLUDED.nombre,
    precio = EXCLUDED.precio;
```

DCL y TCL

DCL: Control de permisos

```sql
GRANT SELECT ON clientes TO analista;
REVOKE DELETE ON clientes FROM analista;
```

TCL: Transacciones

```sql
BEGIN;
UPDATE cuentas SET saldo = saldo - 100 WHERE id = 1;
UPDATE cuentas SET saldo = saldo + 100 WHERE id = 2;
COMMIT; -- o ROLLBACK si algo falla
```

Propiedades ACID

Atomicidad	Todo o nada.
Consistencia	Integridad de datos.
Aislamiento	Transacciones concurrentes no interfieren.
Durabilidad	Los cambios persisten tras fallos.

Contenido de la sección

1. Introducción y Sublenguajes SQL
2. Proceso Interno de una Consulta
3. Joins: Combinando Tablas
4. Subconsultas
5. CTEs
6. Buenas Prácticas y Optimización
7. Hardware y Breve Mirada a NoSQL
8. Ejercicios Resueltos
9. Resumen

Orden Lógico de Ejecución

1. FROM y JOINs
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT (y funciones de ventana)
6. ORDER BY
7. LIMIT / OFFSET

Importante

Los alias de columna definidos en SELECT no pueden usarse en WHERE (porque WHERE se ejecuta antes).

El Camino Físico

1. **Parsing**: Verificación sintáctica y semántica.
2. **Optimización**: Generación de planes de ejecución (basado en costos).
3. **Ejecución**: Interacción con buffer pool y disco.
4. **Retorno**: Envío de resultados al cliente.

EXPLAIN (ANALYZE)

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM clientes WHERE ciudad = 'Madrid';
```

Contenido de la sección

1. Introducción y Sublenguajes SQL
2. Proceso Interno de una Consulta
3. Joins: Combinando Tablas
4. Subconsultas
5. CTEs
6. Buenas Prácticas y Optimización
7. Hardware y Breve Mirada a NoSQL
8. Ejercicios Resueltos
9. Resumen

Tipos de JOIN

INNER JOIN

- Solo filas con coincidencia en ambas.

LEFT JOIN

- Todas las filas de la izquierda + coincidencias.

RIGHT JOIN

- Todas las filas de la derecha + coincidencias.

FULL OUTER JOIN

- Todas las filas de ambas, con NULL donde no hay.

Ejemplos de JOIN

INNER JOIN

```sql
SELECT p.id, c.nombre, p.fecha
FROM pedidos p
INNER JOIN clientes c ON p.cliente_id = c.id;
```

LEFT JOIN (clientes con o sin pedidos)

```sql
SELECT c.nombre, COUNT(p.id) AS num_pedidos
FROM clientes c
LEFT JOIN pedidos p ON c.id = p.cliente_id
GROUP BY c.id, c.nombre;
```

Self-Join y CROSS JOIN

Self-Join: Empleados con su jefe

```sql
SELECT e1.nombre AS empleado, e2.nombre AS jefe
FROM empleados e1
LEFT JOIN empleados e2 ON e1.jefe_id = e2.id;
```

CROSS JOIN: Producto cartesiano

```sql
SELECT * FROM colores CROSS JOIN tallas;
```

Algoritmos Internos de JOIN

Nested Loop: Una tabla pequeña o con índice; va fila por fila.

Hash Join: Tablas grandes sin índice; construye una tabla hash.

Merge Join: Tablas ordenadas; fusiona ambas.

Contenido de la sección

1. Introducción y Sublenguajes SQL
2. Proceso Interno de una Consulta
3. Joins: Combinando Tablas
4. Subconsultas
5. CTEs
6. Buenas Prácticas y Optimización
7. Hardware y Breve Mirada a NoSQL
8. Ejercicios Resueltos
9. Resumen

Tipos de Subconsultas

Escalar (un solo valor)

```
SELECT nombre, (SELECT AVG(precio) FROM productos) AS promedio
FROM productos;
```

Múltiples filas con IN

```
SELECT nombre FROM clientes
WHERE id IN (SELECT cliente_id FROM pedidos WHERE monto > 1000);
```

Subconsultas Correlacionadas

Empleados con salario mayor al promedio de su departamento

```
SELECT e.nombre, e.salario
FROM empleados e
WHERE e.salario > (SELECT AVG(salario)
                   FROM empleados
                   WHERE departamento_id = e.departamento_id);
```

Rendimiento

Se evalúan para cada fila exterior; pueden ser lentas en tablas grandes.

Subconsultas en FROM

Tabla derivada

```sql
SELECT categoria, AVG(precio) AS precio_medio
FROM (SELECT * FROM productos WHERE activo = true) AS productos_activos
GROUP BY categoria;
```

Contenido de la sección

1. Introducción y Sublenguajes SQL
2. Proceso Interno de una Consulta
3. Joins: Combinando Tablas
4. Subconsultas
5. CTEs
6. Buenas Prácticas y Optimización
7. Hardware y Breve Mirada a NoSQL
8. Ejercicios Resueltos
9. Resumen

CTEs Básicas

WITH ...

```sql
WITH ventas_cliente AS (
    SELECT c.id, c.nombre, SUM(dp.cantidad * pr.precio) AS total
    FROM clientes c
    LEFT JOIN pedidos p ON c.id = p.cliente_id
    LEFT JOIN detalle_pedido dp ON p.id = dp.pedido_id
    LEFT JOIN productos pr ON dp.producto_id = pr.id
    GROUP BY c.id, c.nombre
)
SELECT * FROM ventas_cliente WHERE total > 1000;
```

CTEs Múltiples y Recursivas

Múltiples CTEs

```
WITH
    ventas AS (SELECT ...),
    clientes_vip AS (SELECT ...)
SELECT ...
```

Recursiva (organigrama)

```
WITH RECURSIVE organigrama AS (
    SELECT id, nombre, jefe_id, 1 AS nivel
    FROM empleados WHERE jefe_id IS NULL
    UNION ALL
    SELECT e.id, e.nombre, e.jefe_id, o.nivel + 1
    FROM empleados e JOIN organigrama o ON e.jefe_id = o.id
)
SELECT * FROM organigrama ORDER BY nivel;
```

Contenido de la sección

1. Introducción y Sublenguajes SQL
2. Proceso Interno de una Consulta
3. Joins: Combinando Tablas
4. Subconsultas
5. CTEs
6. Buenas Prácticas y Optimización
7. Hardware y Breve Mirada a NoSQL
8. Ejercicios Resueltos
9. Resumen

Consejos de Optimización

- Preferir `EXISTS` a `IN` para subconsultas grandes.
- Evitar funciones en columnas indexadas en `WHERE`.
- Usar `UNION ALL` en lugar de `UNION` si no se necesitan duplicados.
- Evitar `SELECT *` (especificar columnas).
- Cuidado con conversiones implícitas.
- Indexar columnas usadas en `JOIN`, `WHERE`, `ORDER BY`.
- Analizar siempre con `EXPLAIN`.

Ejemplo de Optimización

Consulta ineficiente

```
SELECT * FROM pedidos WHERE EXTRACT(YEAR FROM fecha) = 2025;
```

Versión optimizada (aprovecha índice en fecha)

```
SELECT * FROM pedidos
WHERE fecha >= '2025-01-01' AND fecha < '2026-01-01';
```

Contenido de la sección

1. Introducción y Sublenguajes SQL
2. Proceso Interno de una Consulta
3. Joins: Combinando Tablas
4. Subconsultas
5. CTEs
6. Buenas Prácticas y Optimización
7. Hardware y Breve Mirada a NoSQL
8. Ejercicios Resueltos
9. Resumen

Consideraciones de Hardware

- **Buffer Pool**: Memoria caché de datos e índices (ej. `shared_buffers` en PostgreSQL).
- **SSD vs HDD**: SSD mejora lecturas aleatorias, crítico para índices.
- **Parallel Query**: Uso de múltiples CPUs para consultas pesadas.

Joins en NoSQL

- **MongoDB**: `$lookup` en pipeline de agregación (left outer join).
- **Cassandra**: No soporta joins; modelado desnormalizado.
- **Redis**: No tiene joins; se maneja en aplicación.
- **Neo4j**: Relaciones nativas (navegación en grafos).

Contenido de la sección

1. Introducción y Sublenguajes SQL
2. Proceso Interno de una Consulta
3. Joins: Combinando Tablas
4. Subconsultas
5. CTEs
6. Buenas Prácticas y Optimización
7. Hardware y Breve Mirada a NoSQL
8. Ejercicios Resueltos
9. Resumen

Ejercicio 1: Consultas Básicas

Enunciado

Tabla `empleados` (id, nombre, salario, departamento_id). Insertar tres empleados, actualizar salario de uno, eliminar empleados de un departamento.

Solución

```sql
INSERT INTO empleados (nombre, salario, departamento_id) VALUES
('Juan Perez', 3000, 1),
('Ana Garcia', 3500, 2),
('Luis Fernandez', 3200, 1);

UPDATE empleados SET salario = 3300 WHERE nombre = 'Juan Perez';

DELETE FROM empleados WHERE departamento_id = 2;
```

Ejercicio 2: INNER JOIN y LEFT JOIN

Enunciado

Con las tablas `clientes`, `pedidos`, `detalle_pedido`, `productos`, obtener:

1. Pedidos con nombre de cliente y fecha.
2. Clientes con número de pedidos (0 si no tienen).
3. Total gastado por cliente.

Solución Ejercicio 2

```sql
-- 1. Pedidos con cliente
SELECT p.id, c.nombre, p.fecha
FROM pedidos p INNER JOIN clientes c ON p.cliente_id = c.id;

-- 2. Clientes con numero de pedidos
SELECT c.id, c.nombre, COUNT(p.id) AS num_pedidos
FROM clientes c LEFT JOIN pedidos p ON c.id = p.cliente_id
GROUP BY c.id, c.nombre;

-- 3. Total gastado por cliente
SELECT c.id, c.nombre, COALESCE(SUM(dp.cantidad * pr.precio), 0) AS total
FROM clientes c
LEFT JOIN pedidos p ON c.id = p.cliente_id
LEFT JOIN detalle_pedido dp ON p.id = dp.pedido_id
LEFT JOIN productos pr ON dp.producto_id = pr.id
GROUP BY c.id, c.nombre;
```

Ejercicio 3: Subconsultas

Enunciado

Productos cuyo precio es superior al precio medio de su categoría (subconsulta correlacionada).

Solución

```
SELECT p.nombre, p.precio
FROM productos p
WHERE p.precio > (SELECT AVG(precio)
                  FROM productos
                  WHERE categoria = p.categoria);
```

Ejercicio 4: CTE

Enunciado

Obtener empleados que ganan más que el promedio de su departamento, mostrando nombre, salario y promedio del departamento.

Solución

```sql
WITH promedio_depto AS (
    SELECT departamento_id, AVG(salario) AS salario_prom
    FROM empleados GROUP BY departamento_id
)
SELECT e.nombre, e.salario, pd.salario_prom
FROM empleados e
JOIN promedio_depto pd ON e.departamento_id = pd.departamento_id
WHERE e.salario > pd.salario_prom;
```

Ejercicio 5: Optimización

Consulta lenta

```
SELECT * FROM pedidos WHERE EXTRACT(YEAR FROM fecha) = 2025;
```

Optimización

- Reescribir: `WHERE fecha >= '2025-01-01' AND fecha < '2026-01-01'`
- Crear índice en `fecha`.
- Verificar con `EXPLAIN ANALYZE`.

Ejercicio 6: CTE Recursiva

Enunciado

Obtener organigrama completo con niveles desde `empleados(id, nombre, jefe_id)`.

Solución

```sql
WITH RECURSIVE organigrama AS (
    SELECT id, nombre, jefe_id, 1 AS nivel
    FROM empleados WHERE jefe_id IS NULL
    UNION ALL
    SELECT e.id, e.nombre, e.jefe_id, o.nivel + 1
    FROM empleados e JOIN organigrama o ON e.jefe_id = o.id
)
SELECT * FROM organigrama ORDER BY nivel, nombre;
```

Contenido de la sección

1. Introducción y Sublenguajes SQL
2. Proceso Interno de una Consulta
3. Joins: Combinando Tablas
4. Subconsultas
5. CTEs
6. Buenas Prácticas y Optimización
7. Hardware y Breve Mirada a NoSQL
8. Ejercicios Resueltos
9. Resumen

Conceptos Clave de la Sesión 5

- **Sublenguajes SQL**: DDL, DML, DCL, TCL.
- **Transacciones**: ACID, COMMIT, ROLLBACK, SAVEPOINT.
- **Orden lógico de ejecución** y su impacto.
- **Joins**: INNER, LEFT, RIGHT, FULL, CROSS, self-join.
- **Subconsultas**: escalares, correlacionadas, en FROM.
- **CTEs**: simples, múltiples, recursivas.
- **Optimización**: índices, evitar funciones en WHERE, usar EXISTS, analizar con EXPLAIN.
- **Hardware**: buffer pool, SSD, parallel query.
- **NoSQL**: aproximación a joins ($lookup, desnormalización).

¡Gracias por su atención!